CHAPTER 1

INTRODUCTION

1.1 Overview

A Trojan is a harmful and unauthorized alteration inserted in the design or implementation of an integrated circuit. Trojans are typically concealed and are designed to be activated under abnormal conditions, thereby being extremely hard to detect by means of conventional testing methods. When activated, they have the ability to leak sensitive information, reduce performance, or even cause complete destruction of the system. In high reliability domains such as defence, medicine, and finance, the discovery of a Trojan can lead to disastrous security breaches, data corruption, or loss of control over the system. Therefore, its detection and mitigation have emerged as key elements of hardware security. The International Energy Agency (IEA) projects a significant development within the number of electric cars on the street, evaluating that the worldwide armada seem reach 145 million by 2030 on the off chance that governments seek after arrangements that advance a green recuperation from the COVID-19 emergency. This projection underscores the criticalness of creating effective charging foundation to bolster the extending electric vehicle advertise. Charging foundation plays a pivotal part in encouraging the widespread selection of EVs by tending to run uneasiness and empowering helpful charging for vehicle proprietors.

1.2 Background

The RISC-V instruction set architecture shown in Figure 1.1 has been of interest due to its open-source nature, modularity, and expandability. This makes it possible for one to implement general-purpose processors that can be customized to fit different applications, ranging from research to commerce. This openness comes with the disadvantage of being open to new forms of security attacks, such as Trojan attacks. These are illicit changes that may be covertly added at any stage of the hardware design cycle. They can remain dormant until they are triggered and are difficult to detect through traditional methods. Therefore, ensuring robust hardware security mechanisms has become essential to safeguard RISC-V–based systems against such emerging threats.

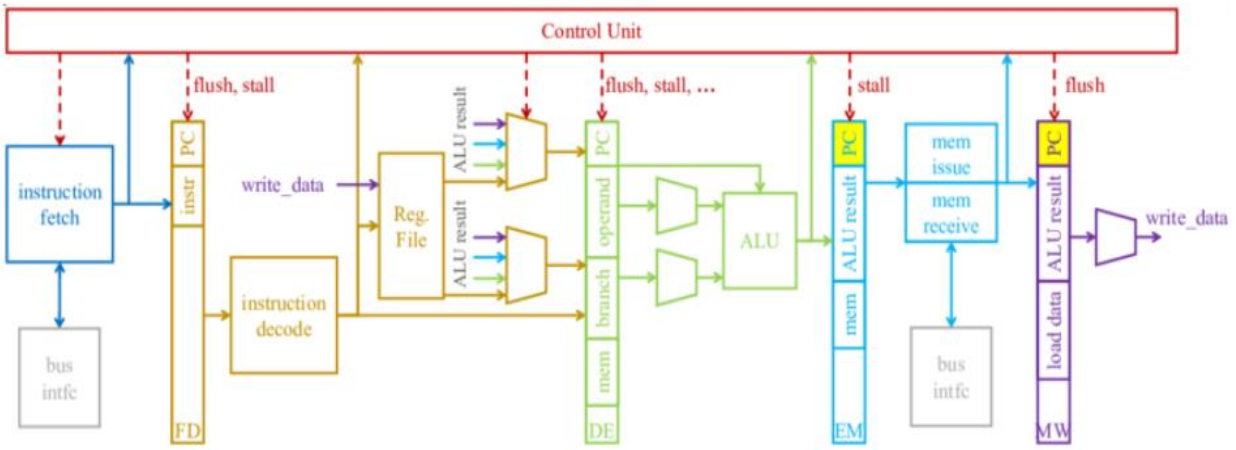

Figure 1.1: Block Diagram of RISC-V Processor

With System-on-Chip designs becoming increasingly complex and semiconductor manufacturing being globalized, it has become increasingly challenging to provide hardware trust. Trojans can be inserted into the design at any point from RTL design to physical implementation without any obvious evidence. These are difficult to detect as they may remain dormant until they are triggered, which happens under certain conditions. These are hard to detect using traditional simulation or testbench methods. In this situation, side channel analysis methods such as power monitoring are becoming popular due to the use of physical characteristics of the system (such as power consumption, timing, or electromagnetic emissions) to detect irregularities that might indicate the presence of a hidden Trojan.

1.3 Problem Statement

With the rapid growth of open-source architectures such as RISC-V, hardware designs are increasingly exposed to third-party IP integration and distributed development environments. This openness, while beneficial for innovation and customization, significantly increases the possibility of malicious hardware Trojans being inserted at various stages of the design, synthesis, or fabrication flow. These Trojans are engineered to remain inactive under normal functional testing, making them extremely challenging to detect through traditional verification or structural analysis techniques.

Among the various side-channel indicators, power consumption is one of the most sensitive parameters that can reveal hidden malicious activity. However, reliably activating these dormant Trojans and capturing their impact in real time remains a major challenge. Existing methods often depend on pre-characterized patterns or offline analysis, which fail to detect Trojans that require highly specific trigger conditions or dynamic runtime scenarios.

Therefore, there is a need for a robust, real-time power profiling approach capable of stimulating potential Trojan trigger conditions and identifying subtle variations in power signatures as they occur. This project addresses this challenge by developing a methodology to activate hardware Trojans using real-time power profiling, enabling early detection and improving the overall security and trustworthiness of hardware systems.

1.4 Literature Survey

1. **Lamech, C., Rad, R. M., Tehranipoor, M., & Plusquellic, J. (2011). "An experimental analysis of power and delay signal-to-noise requirements for detecting Trojans and methods for achieving the required detection sensitivities". IEEE Transactions on Information Forensics and Security, 6(3), 1170-1179.**

The document presents an approach based on unsupervised learning for the detection of Trojans in FPGA systems based on the side-channel characteristics of the system such as power consumption and signal activity. The approach utilizes techniques such as k-means and hierarchical clustering in order to differentiate between clean and infected circuits without the need for a labeled data set. The approach is non-invasive, scalable, and has low false positive rates, making it well suited for scenarios where labelled data sets are scarce or unavailable. Also, towards the same goal of effective Trojan detection, this work has undertaken an alternative technical approach towards implementing real-time monitoring of RTL level signals. While not analysing the side channel data in execution, as done by other works, this approach has emphasized on live signal comparison between clean and infected modules in order to exhibit their behavioural differences. This not only reduces the computationally intensive requirements but also paves the way for faster and direct detection, making the feasibility of Trojan defence in real-time embedded systems much more practical.

2. **Rathor, V. S., Podder, S., & Dubey, S. (2025). "An ensemble learning model for Trojan detection in integrated circuit design". Computers and Electrical Engineering, 123, 110090.**

   This work presents an ensemble learning model which uses decision trees, support vector machines (SVMs) and random forests for Trojan detection in integrated circuit design. The proposed ensemble learning model uses the simulation-based power traces and benefits from the strengths of different classifiers to achieve better detection accuracy in comparison to a single model. Using different classifiers also makes this solution more resilient against different types of Trojans and methods of insertion. This ensemble solution offers a holistic solution to hardware security. However, the ensemble model incurs heavy computational resource requirements in both training and inference phases due to the complexities involved in managing multiple classifiers and dealing with large simulation data.

3. **Sunkavilli, S., Zhang, Z., & Yu, Q. (2021, July). "New security threats on FPGAs: From FPGA design tools perspective". In 2021 IEEE Computer Society Annual Symposium on VLSI (ISVLSI) (pp. 278-283). IEEE.**

   With the increasing use of FPGAs in cloud, machine learning, and high performance computing applications, security threats to FPGAs also become more complex. Most of the existing research has focused on various threats to a standalone FPGA system such as counterfeiting and Trojans. However, with increasing availability of cloud-FPGA services and open-source design tools, new threats have emerged such as vulnerabilities in the FPGA design tools and collaboration environment itself, despite the low attention paid to these issues. In this work, we detect Trojans in FPGA designs by analysing real-time monitoring of Verilog modules and understanding their potential behavioural anomalies that could indicate the presence of a Trojan. Our framework for Trojan detection based on runtime anomalies is practical and scalable.

4. **Elnaggar, R., Chaudhuri, J., Karri, R., & Chakrabarty, K. (2022). "Learning malicious circuits in FPGA bitstreams". IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 42(3), 726-739.**

In this work, we propose to identify Trojans at the bitstream level by applying machine learning methods to represent malicious circuits as data series extracted from bitstreams, and then apply supervised classifiers such as random forest and SVM for accurate classification of malicious and legitimate designs. This work provides a new approach to identifying potential security threats in FPGA designs prior to their deployment.

5. **Chaudhuri, J., & Chakrabarty, K. (2022, May). "Detection of malicious FPGA bitstreams using CNN-based learning". In 2022 IEEE European Test Symposium (ETS) (pp. 1-2). IEEE.**

   This paper models the problem of Trojan detection in multi-tenant FPGA environments as one of identifying anomalies in data series representing bitstreams and shows strong results using Convolutional Neural Networks (CNNs). This work demonstrates the potential of deep learning techniques for embedded hardware security at the bitstream level.

6. **Cruz, J., Posada, C., Masna, N. V. R., Chakraborty, P., Gaikwad, P., & Bhunia, S. (2023). "A framework for automated exploration of trojan attack space in FPGA netlists". IEEE Transactions on Computers, 72(10), 2740-2751.**

   This study reports a methodology to insert and evaluate various set of Trojans in the FPGA netlists to understand the stealth, power and delay impacts of these benchmarks. We extend this work to define and evaluate 'dark silicon' Trojans which do not have any area or power overhead. This work helps to understand the attack vectors and the stealth of these Trojans in various predeployment evaluations. The capability to realize these effects helps us better understand the behavior of Trojans in FPGA systems and the ensuing security threats.

7. **Zahid, K. (2022). The detection of malicious modifications in the FPGA. Journal of Electronic Testing, 38(3), 247-260.**

   The author has proposed a framework which is based on using the Natural Language Processing (NLP) technique to determine the syntactic patterns present in the FPGA bitstream files such as XDL and NCD for the detection of malicious modifications. This method is based on the structural components of the bitstream to determine the changes that have occurred in the system which might lead to the existence of a Trojan or any other malicious modification. This method provides security

to the FPGA designs by adding an extra layer of security before the deployment of the system although this method is useful for pre-deployment assessments, our hardware-based runtime monitoring approach can be used along with this method which is focused towards finding the functional disruptions during the run-time of the system to provide a more secure framework.

8. **Kumar, B., Prasanna, S., Bharath, P., & Joice, C. S. (2023). FPGA Trojan attack impacts on architecture performance: a systematic approach revealed. In 2023 8th International Conference on Communication and Electronics Systems (ICCES) (pp. 128-132).**

In this paper, we focus on runtime performance metrics and adopt the Linear Pattern Convergence Model to detect the anomalies caused by Trojans. With on-chip monitoring, the approach can be trained to detect anomalies without altering the fundamental microprocessor architecture. This way, we can run the method smoothly on current systems and keep on observing the systems behaviour on run-time. The method is dependent on real-time performance data to detect the security threat present in the system. Hence, the method can identify Trojan anomalies in the system that can be detected quickly. The approach is designed in such a way that it is not intrusive to the system and can be used on various systems while providing a high efficiency. This method is similar to our project as well which is also designed to find the disruptions but in a different approach i.e. by monitoring the signal and power levels to the FPGA platform for the real-time detection of functional anomalies.

1.5 Objectives

This project aims to develop a Trojan detection method using power analysis methods within an FPGA-based RISC-V processor.

The objectives of our project are:

1. Understand the potential vulnerabilities of the ALU in a RISC-V processor.
2. Design a hardware-based detection system for malicious logic in ALU operations.
3. Integrate the detection mechanism on a hardware platform using the Basys3 FPGA board.
4. Capture bit-level switching activities and store them in real time.

5. Detect the anomaly in power usage by using RTL-based detection method.

Building on these objectives, the project also aims to establish a comprehensive framework that not only captures real-time switching activity but also correlates this information with fine-grained power variations within the FPGA-based RISC-V system. This involves generating diverse operational scenarios for the ALU to observe how different instruction patterns influence both dynamic and leakage power under normal and Trojan-injected conditions. The system will incorporate advanced trace-processing methods, including filtering, normalization, and anomaly scoring, to enhance the sensitivity of Trojan detection. Additionally, the project seeks to create a controlled environment for injecting and activating hardware Trojans to study their behavior across varying trigger conditions, payload types, and execution contexts. Emphasis is placed on ensuring that the detection mechanism introduces minimal hardware and timing overhead, preserving the processor's functional correctness and performance. The overall goal is to produce a robust, repeatable, and hardware-validated methodology that can serve as a reference model for future research in FPGA-based hardware security, ultimately strengthening trust in open-source architectures like RISC-V.

1.6 Motivation

The evolution of modern computing systems has led to an unprecedented reliance on hardware components whose design and fabrication often involve multiple untrusted entities. As supply chains grow more complex, the possibility of hardware tampering has become a critical concern, particularly in open-source architectures like RISC-V, where accessibility and modularity—though beneficial—also provide opportunities for adversaries to insert hidden malicious circuitry. Hardware Trojans represent one of the most dangerous forms of such threats because they can be deeply embedded within internal modules like the ALU, remaining inactive during normal operation and bypassing almost all traditional verification techniques. Their selective activation, triggered only by rare events or specific instruction patterns, enables them to leak data, alter system behavior, or degrade performance without leaving obvious functional traces.

Implementing such a detection mechanism on a RISC-V processor mapped to the Basys3 FPGA provides not only a practical, hands-on validation platform but also contributes to the broader

research goal of improving trust in open-source hardware ecosystems. By developing a robust detection methodology grounded in real switching activity, this project aims to support the creation of trustworthy, secure, and verifiable hardware systems capable of withstanding the emerging threats in today's complex digital landscape.

The Arithmetic Logic Unit (ALU) is perhaps the most critical component of a processor, responsible for basic arithmetic and logical operations that directly influence the outcome of every instruction. Because of the central position of the ALU in the processor's data path, even a FPGA-Driven Power Analysis Detecting Trojan Threat In RISC-V Processor, ACED 3 slight deviation in the operation of the ALU by design or by fault can result in incorrect computation, data corruption, and system breakdown. The ALU, shown in Figure 1.2, is therefore a prime candidate for stealthy changes in logic that do not affect every computation, but can surreptitiously destroy program execution under specific conditions.

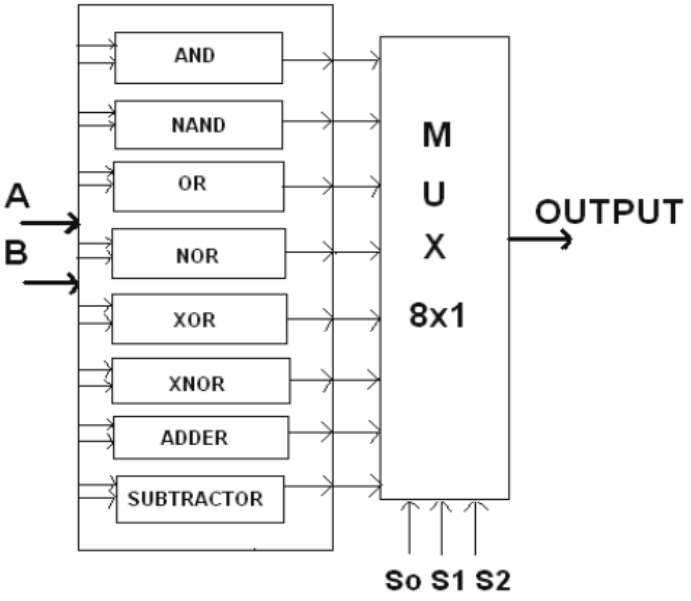

Figure 1.2: Simple block diagram of an ALU

1.7 Existing Systems

1. Conventional Functional Testing

Most existing hardware verification relies on functional simulation and testbenches to check whether the circuit behaves correctly. However, hardware Trojans are intentionally designed to remain inactive during normal or conventional test patterns. As a result, functional testing often fails to trigger the Trojan and cannot detect hidden malicious logic inside critical components like the ALU.

2. Offline Power Analysis Techniques

Some detection methods use power traces collected from simulations or external measurement hardware such as oscilloscopes. While this helps identify abnormal power patterns, these techniques are performed offline and lack real-time responsiveness. They also struggle to detect Trojans that activate under specific runtime conditions or rare event sequences that simulations may not capture.

3. Structural and Layout-Based Inspection

Traditional hardware security approaches include gate-level netlist comparison, layout inspection, and design-for-trust techniques. These methods require access to golden reference models or chip-level layout files, which are often unavailable—especially in open-source or third-party IP environments. Moreover, structural inspection cannot always identify small Trojans intentionally designed to blend into the circuit.

1.8 Proposed System

The proposed project introduces a real-time, FPGA-implemented Trojan activation and detection framework that focuses on the dynamic switching activity and power consumption of the RISC-V processor's ALU. Unlike traditional approaches, this method operates directly on hardware, providing superior accuracy, responsiveness, and applicability to real-world conditions.

1. Real-Time Activation Through ALU Workloads

The system executes specifically crafted instruction patterns on the RISC-V processor mapped onto the Basys3 FPGA. These instruction sequences are designed to stimulate rare conditions that may

activate hidden Trojan logic. By targeting ALU operations where arithmetic and logical processing involves significant bit-level transitions—the system increases the probability of triggering Trojans embedded in the data path or control logic.

2. Continuous Power Profiling Integrated into RTL

A custom RTL-based power monitoring mechanism is embedded within the hardware architecture to capture cycle-by-cycle switching activity. Instead of relying on external oscilloscopes, the system uses internal counters and sensors to measure toggling activity, enabling on-chip power estimation. This ensures the detection process is highly responsive, real-time, and immune to external measurement noise.

3. Switching Activity–Based Anomaly Detection

The proposed system analyses changes in bit-level transitions within the ALU and correlates them with expected power behaviour. Hardware Trojans, once activated, alter switching patterns or introduce additional logic transitions that manifest as abnormal or inconsistent power signatures. The system uses signal-processing and statistical analysis on the switching trace to isolate these anomalies from normal instruction-induced variations.

FPGA Validation Using Basys3

The use of a Basys3 FPGA offers several advantages:

1. Rapid prototyping and reconfigurability
2. Real-time observation of hardware behavior
3. Ability to introduce controlled Trojan-inserted designs
4. Direct experimentation on RISC-V ALU operations

This makes the proposed system a practical, hardware-validated platform for Trojan research.

Eliminates Need for Golden Models

Unlike structural verification methods, this detection approach does not require a golden reference design. Instead, anomaly detection is based purely on real-time power and switching activity, allowing detection even in environments where trusted reference models are unavailable.

Robust and Scalable Security Framework

The proposed system aims to deliver a scalable and deployable methodology that can be extended to:

1. Other processor cores
2. Additional data path components
3. Different FPGA boards
4. ASIC-level hardware security applications

Ultimately, the system enhances hardware trustworthiness by providing an accurate, real-time, and FPGA-validated approach for detecting malicious Trojans in RISC-V processors

1.9 Related work

Hardware Trojan detection has been an active research area over the past decade, driven by the increasing globalization of semiconductor manufacturing and the growing reliance on third-party IP cores. Early research primarily focused on functional testing techniques, where test vectors were generated to maximize coverage of internal nodes. However, studies revealed that Trojans are deliberately engineered with extremely rare trigger conditions, making functional activation nearly impossible and exposing the limitations of purely logic-based verification.

To overcome these shortcomings, researchers explored side-channel analysis**,** particularly power and delay measurements, as indicators of hidden malicious circuits. Among these, power analysis emerged as one of the most promising approaches, as even small Trojans tend to cause subtle changes in switching activity. Several works demonstrated the feasibility of using power signatures to differentiate Trojan-free hardware from infected designs. However, many of these techniques relied on offline measurement platforms, such as oscilloscopes or external current probes, which

introduced noise, lacked real-time capability, and were unsuitable for embedded systems or runtime detection.

Another major stream of research involved Golden Model–based comparison, where a trusted reference design is compared with the suspect implementation at the gate, layout, or functional level. While effective for small designs, researchers noted that this approach becomes impractical for large or commercially sourced IPs, where reference models are unavailable. Furthermore, highly stealthy Trojans can blend into the circuit without significant changes in the structure, making detection extremely difficult even with detailed netlist analysis.

Recent studies have begun to shift toward FPGA-based prototyping for Trojan detection, leveraging reconfigurability and real-time monitoring capabilities. Work in this direction has shown that internal switching activity and on-chip power estimation can provide more accurate and noise-resistant insights than external measurement methods. Some researchers developed real-time switching counters and power estimation models, proving that physical side-channels can be monitored directly within RTL logic. However, existing implementations often focused on simple benchmark circuits or synthetic triggers, lacking evaluation on practical architectures like RISC-V processors.

Additionally, emerging research highlights the importance of activity correlation methods, where bit-level transitions within data path modules particularly ALUs are analysed to identify anomalies introduced by malicious logic. Studies have emphasized that Trojans embedded inside ALUs can alter operand flow, modify control signals, or introduce additional toggling when activated, making these components high-value targets for runtime detection.

Despite the progress, there remains a clear gap in the literature: a fully integrated, real-time Trojan activation and detection framework implemented directly on FPGA hardware for a RISC-V processor. Most existing work is either theoretical, simulation-based, or dependent on external equipment. This motivates the need for a practical, FPGA-validated system capable of detecting

anomalies through real-time switching and power profiling—an area directly addressed by the current project.

1.10 Contributions

1. FPGA-Based Real-Time Trojan Detection Framework

A complete hardware-implemented detection methodology is developed on the Basys3 FPGA, enabling real-time observation of switching activity and power variations in a RISC-V processor rather than relying on offline tools or simulations.

2. ALU-Centric Trojan Activation Approach

The project introduces a targeted activation mechanism focusing on ALU operations, which are highly sensitive to bit-level transitions. This approach increases the likelihood of triggering stealthy Trojans embedded within arithmetic and logic data paths.

3. Switching Activity Monitoring at RTL Level

A custom RTL-based power estimation logic is designed to capture fine-grained toggling signals at runtime. This enables precise correlation between internal switching behavior and potential Trojan activation events.

4. Dynamic Power Anomaly Detection Technique

The project proposes a robust anomaly detection method that compares expected and observed power patterns, allowing identification of deviations caused by malicious circuits with minimal performance overhead.

5. Hardware Validation Using RISC-V Core

Unlike many simulation-driven studies, this work validates Trojan detection in a real hardware environment using a functioning RISC-V processor architecture, demonstrating practical feasibility and reliability.

6. Scalable Methodology for Hardware Security Analysis

The framework developed in this project can be extended to other processor components, different FPGA boards, and larger system architectures, providing a foundation for future research on side-channel-based Trojan detection.

7. Reduced Dependency on Golden Models

The detection system operates without requiring a trusted golden reference design, addressing a major challenge in modern hardware supply chains where reference models are often unavailable or incomplete.

CHAPTER 2

IMPLEMENTATION

2.1 Block Diagram

The block diagram of the FPGA is shown as below:

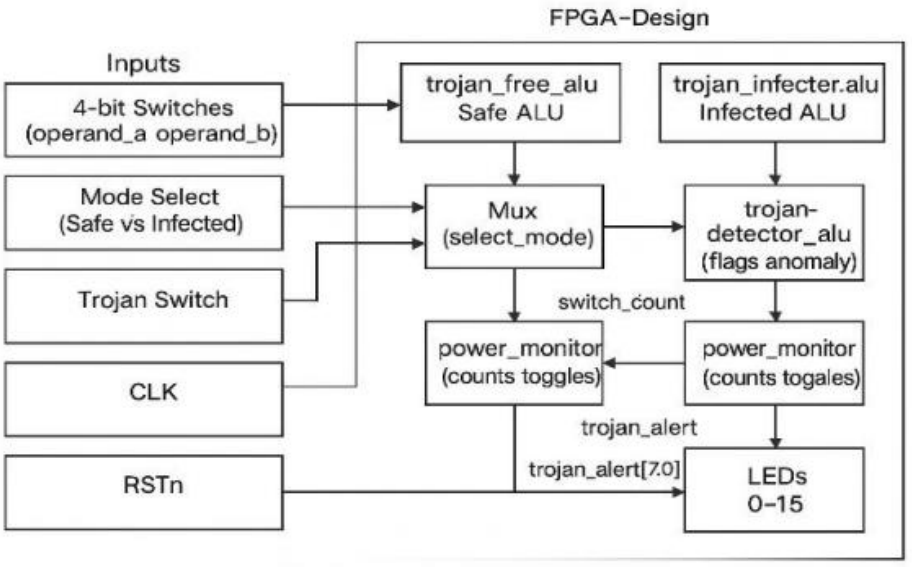

Figure 2.1: Block Diagram

The Figure 2.2 shows the block diagram of the project implemented on the Basys 3 FPGA board. There are two modules implemented inside the FPGA, one represents the Trojan free (safe) logic and other represents the Trojan infected logic. Both the modules take input signal as their input, perform their respective logic and give output. MUX is used to select between the two modules based on the select mode signal. The output selected by the MUX is further observed by a respective detection unit. The detection unit constantly compares the behaviour of the selected output module

with the expected behaviour. Any irregularity found by the comparator triggers the alert signal which is indicated by the LEDs on the board. In addition to the runtime controls like the LEDs, switches are also provided for manual controlling the Trojan (trojan switch) and module selecting (select mode). This block level design allows the real time observation of Trojan during the operation of the system. This project was implemented without changing the internal FPGA structure.

In addition to the module selection and anomaly comparison, the block diagram also incorporates a power monitoring subsystem that plays a crucial role in detecting hardware Trojan activity. Both the safe ALU and the infected ALU are connected to dedicated power_monitor blocks, which continuously count the number of bit-level toggles occurring during each operation. Since switching activity is directly proportional to dynamic power consumption, these toggle counts act as a real-time power signature of the executing module.

The switch_count signals generated by the power monitors are fed to the detection unit, where they are analyzed for irregularities or deviations from the expected toggle pattern. The Trojan detector uses these switch_count values to determine whether the ALU is consuming abnormal power, which may indicate that hidden malicious logic has been activated. When the toggle count exceeds a defined threshold or exhibits unexpected behavior, the detection block raises a trojan_alert signal.

This alert is then mapped to the LED outputs (0–15) on the Basys3 board, allowing the user to visually observe when suspicious activity is detected. The LEDs serve as an immediate hardware-level indication of anomaly detection, enabling real-time security feedback without requiring an external monitoring setup.

Additional input controls such as the trojan_switch allow the user to manually activate the Trojan logic within the infected ALU, which helps in testing and validating the detection mechanism. The mode select signal provides the ability to switch between safe and infected modules, enabling comparative analysis under identical input conditions. The clock (CLK) drives the entire design, ensuring synchronous operation, while the reset (RSTn) signal initializes the system to a known state before execution begins.

Overall, this block diagram represents a complete and self-contained experimental setup where both functional outputs and power signatures are monitored in parallel. By observing real-

time switching activity and comparing it against expected patterns, the system enables accurate detection of Trojan activation without altering the FPGA's internal architecture. The modular design also makes it easier to extend, modify, and analyse different Trojan behaviours within a controlled hardware environment.

2.2 Methodology

This project is focused on a power-analysis detection technique which observes the switching activities of the ALU in a RISC-V processor which is synthesized and implemented on an FPGA. The hypothesis behind this technique is that when a Trojan is activated within the processor, it induces changes in the logic activities of the design, which results in an increase in the number of bit transitions. Consequently, the switching activities of the design also rise, and the dynamic power consumption of the design is also raised. This dynamic power consumption can be seen and quantified at the RTL level. The given technique can be categorized into the following phases:

1. ALU Design and Integration

An Arithmetic Logic Unit (ALU) is a part of a computer system that performs basic arithmetic and logical operations. The ALU was coded using Verilog Hardware Description Language (HDL) and in two forms. The first one is a validated or tested version of the ALU, which performs basic operations like addition, subtraction, AND, OR, and XOR between two 4-bit operands (operand_a, operand_b) based on a 4-bit control signal (alu_sel).

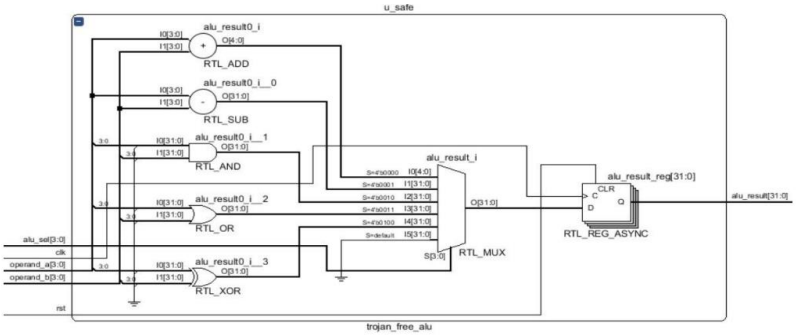

Figure 2.2: Trojan Free ALU Circuit

The above diagram shows the clean or trusted version of the ALU which performs basic addition, subtraction, AND, OR, XOR operations on two 4-bit operands (operand_a, operand_b) based on the 4-bit control signal (alu_sel). A multiplexer selects the output of the ALU based on the value of alu_sel. The output of the ALU is passed through a register for clock synchronization. This version of the ALU only performs the operations as described earlier and does not contain any malicious codes.An infected or Trojan version of the ALU is implemented which is similar to the clean version of the ALU.

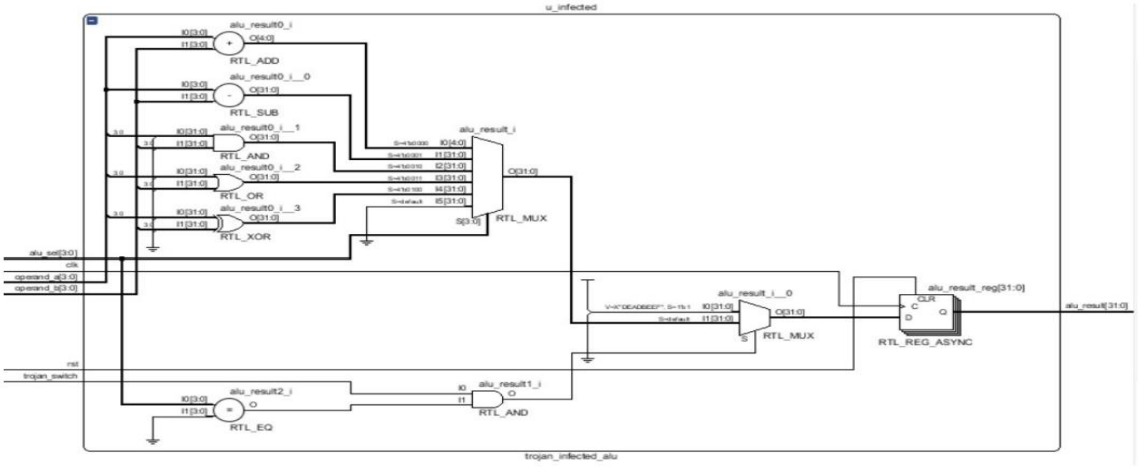

Figure 2.3: Trojan-Induced ALU Circuit

The above schematic shows the infected or Trojan version of the ALU which is similar to the clean version of the ALU. An infected or Trojan version of the ALU is implemented which is chosen to simulate the real behaviour of a Hardware Trojan. A Hardware Trojan is embedded inside the ALU which is programmed to execute when a specific rare input condition is met.

2. Switching Activity Capture and Power Monitoring

To identify unusual transitions, internal signals from the ALU were monitored and processed through a bitwise XOR logic to detect switching events as shown . A population count (popcount) circuit was employed to tally the number of bits that toggled between successive clock cycles. This tally indicates the dynamic switching activity of the ALU. population count circuit was employed to tally the number of bits that toggled between successive clock cycles. This tally indicates the dynamic switching activity of the ALU.

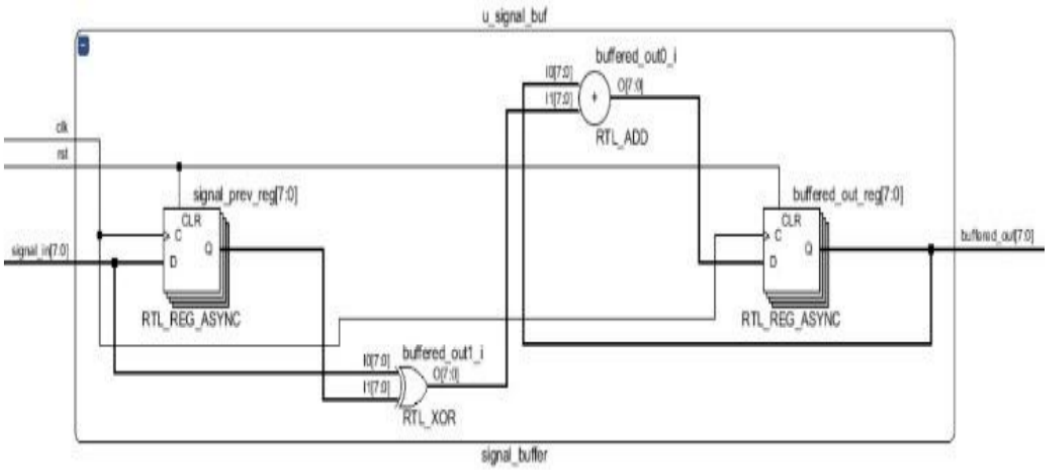

Figure 2.4: Signal Buffer for Switching Activity

In this project, the power monitoring is done indirectly by counting the switching activity in the ALU as it is directly related to the dynamic power consumption as shown in the figure 3.3. Instead of using outside instruments the FPGA monitors the bit-level transition of certain signals over clock cycles. A higher count of transitions indicates higher power consumption and hence possibly the activation of the Trojan.

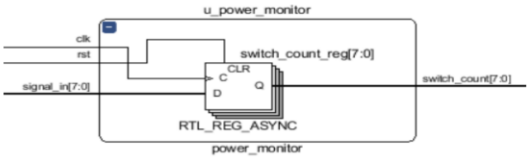

Figure 2.5: Power Monitor

The power monitor counts the bit-level changes in the output of ALU using signal_buffer.v it compares the current and previous value and updates the value of toggle count. The switching activity of the unusual power behavior of the ALU helps to identify the presence of Trojan circuit.

3. Threshold-Based Detection Logic

The count of switching activities was compared against a predetermined threshold. When the Trojan is in dormant state the switching count remains within a predetermined range. When the Trojan is activated it causes an irregular increase in the switching count which is more than the threshold. This will trigger an alert signal inside the system which signifies the presence of a threat.

2.3 Schematic Diagram

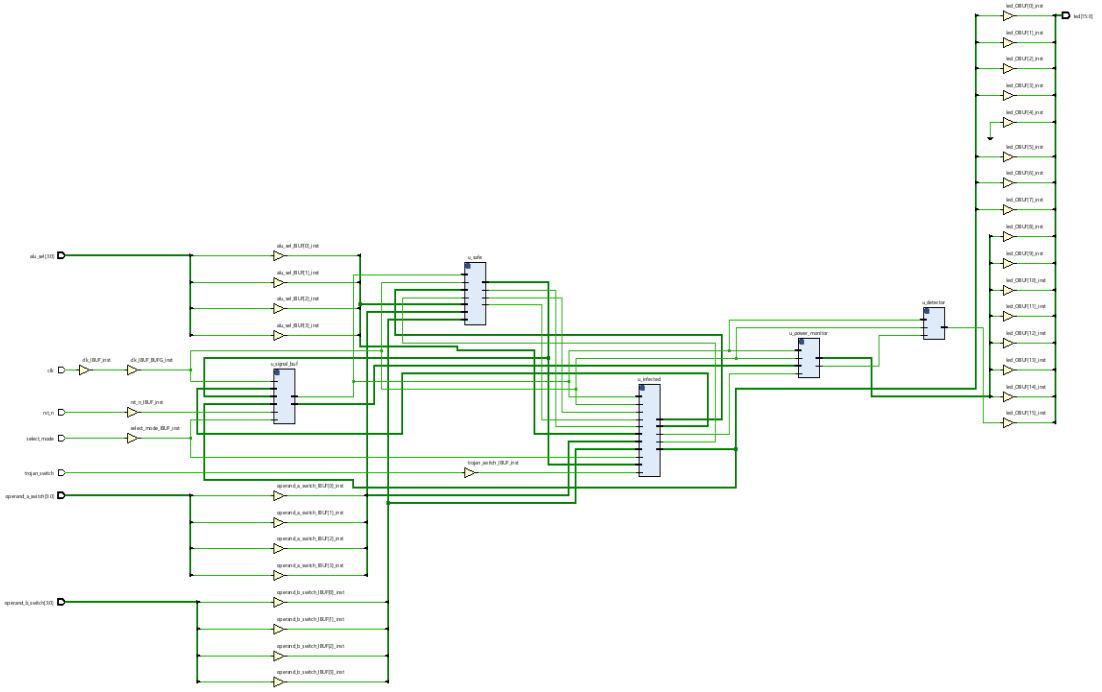

Figure 2.6: Schematic Diagram

The overall system is implemented using a hierarchical and modular digital design methodology, comprising several interlinked functional blocks that collectively manage signal acquisition, processing, monitoring, and output control. The schematic receives multiple input sources, including system clock, reset, mode-selection signals, sensor data buses, and SPWM (Sinusoidal Pulse Width Modulation) reference inputs. Each input is first conditioned through dedicated buffering and synchronization circuits to ensure signal integrity and timing alignment before entering the core processing units.

At the heart of the design, key modules include the SPWM processing unit, safety and protection module, internal controller (FSM-based), current-monitoring module, and detector block. The SPWM processing unit generates precise pulse-width modulated signals based on input references and operational modes, while the safety and protection module continuously evaluates operational limits, fault conditions, and abnormal scenarios, feeding corrective actions back to the internal controller. The current-monitoring module measures real-time currents from the load or motor, providing critical feedback to prevent overcurrent situations, and the detector block assists in evaluating system states and fault detection.

These modules communicate through multi-bit buses and control signals, allowing coordinated operation such as PWM modulation, current regulation, mode-dependent responses, and fault handling. The internal controller orchestrates the timing, sequencing, and decision-making logic, leveraging input feedback to maintain system stability and safe operation across different modes. Processed outputs from the SPWM, monitoring, and control units are routed to designated output interfaces on the schematic, where signals are mapped to external pins for driving motors, indicating system status, and executing control commands.

This structured design emphasizes modularity, signal integrity, and fault-resilient operation, facilitating scalability, easier debugging, and reliable functionality under a range of operational conditions. The hierarchical arrangement of blocks ensures that each functional unit can operate semi-independently while maintaining proper coordination through well-defined interfaces, resulting in a robust and maintainable digital system architecture.

CHAPTER 3

SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

The hardware requirements are:

- FPGA Basys 3 Board

3.2 SOFTWARE REQUIREMENTS

The software requirements are:

- Programming language: Verilog HDL
- Tools: Xilinx Vivado

3.3 HARDWARE COMPONENT

FPGA Basys 3

FPGAs are integrated circuits that are programmed to perform a particular hardware function after fabrication. They are suitable for real-time testing and one-off digital designs due to their reconfigurable nature. As opposed to the time-consuming and expensive traditional ASIC design, FPGAs can be debugged in real time, provide quick iteration cycles, and offer exceptional observability, all of which are important features when designing detection systems for logic anomalies.

For this project, we will use the Basys 3 FPGA board, which features the Xilinx Artix-7. It is ideal for educational and research-oriented development activities. Furthermore, FPGAs offer unparalleled design flexibility because of their reconfigurability; designers Also, the Basys 3 FPGA, with its multiple I/O pins, integrated clocking resources, and support for high-speed simulation and

analysis, facilitates exhaustive normal and altered processor configuration testing. This project demonstrates how design methodologies based on FPGAs can be developed and analysed in a short timeframe, thus enabling rapid evaluation of meticulous hardware security frameworks in actual

The RISC-V Processor is designed and implemented within the project, with the FPGA acting as the platform and design environment to perform the power consumption pattern monitoring on the RISC-V processor. FPGAs have the following benefits to projects related to security:

1. Parallelism: One can look at many signals at once.
2. Reconfigurability: It is easy to switch logic for different test conditions.
3. Control of signal paths and monitoring circuits is what is meant by low-level control.

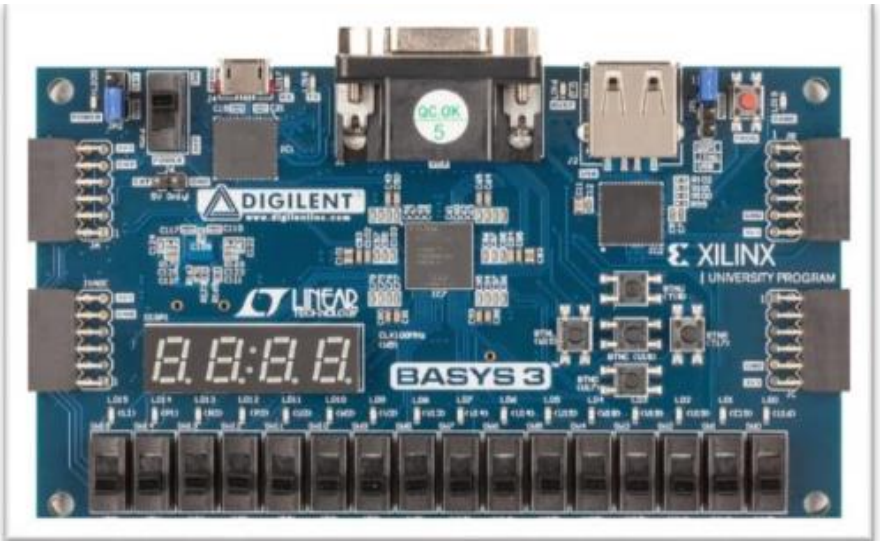

Figure 3.1 FPGA Basys-3

The Basys 3 FPGA board is a versatile educational and development platform built around the Xilinx Artix-7 XC7A35T FPGA, designed to help students and engineers learn digital logic and implement hardware designs. The board provides a variety of built-in components that make it easy to test and observe digital circuits without needing external hardware. It includes a 100 MHz clock oscillator that serves as the main timing reference for synchronous designs. To supply user inputs, the board has sixteen slide switches and five push-buttons that can be used to control logic, change modes, or trigger operations inside the FPGA. For visual output, it offers sixteen LEDs and four

multiplexed seven-segment displays, which allow users to monitor internal signals, display numbers, or show system states directly on the board. Communication with a computer is handled through a USB connection that supports both JTAG programming and UART serial communication, making it convenient to load bitstreams and interact with running designs. The board also includes PMOD connectors, which act as general-purpose expansion ports for connecting external modules such as sensors, communication interfaces, or custom circuits. Some Basys 3 variants also provide VGA and audio interfaces, allowing projects involving video signals or sound generation. The board is powered through USB or an external supply, and includes onboard voltage regulators to provide stable power to the FPGA and peripherals. Overall, the Basys 3 board offers a compact, well-integrated environment that helps learners and developers easily experiment with digital circuits, embedded systems, and FPGA-based applications.

3.4 SOFTWARE COMPONENTS

Xilinx Vivado

Xilinx Vivado is an advanced FPGA design and development environment created by Xilinx to support modern devices such as the Artix-7, Kintex-7, Virtex-7, Zynq, and later series. It replaces the older ISE toolchain and introduces a more powerful, high-performance workflow for building complex digital systems. Vivado allows engineers to write and manage hardware designs using languages like Verilog, VHDL, or System Verilog, and it integrates all essential stages of FPGA development into one platform. The software includes a feature-rich text editor for writing HDL, simulation tools for checking logical behavior before implementation, and a synthesis engine that translates the HDL description into low-level logic elements optimized for the target FPGA. After synthesis, Vivado performs place-and-route, where it maps the design onto the physical resources of the FPGA, ensuring that timing constraints, clock requirements, and routing paths are correctly satisfied. One of its major strengths is the block design environment, where users can graphically build systems using pre-designed intellectual property blocks, processors, memory controllers, communication interfaces, and custom modules. Vivado also provides detailed timing analysis, power estimation, resource usage reports, and debugging utilities such as the Integrated Logic Analyzer, which allows designers to capture internal signal activity from a running FPGA. Through

the Hardware Manager, users can program the FPGA directly, test designs in real time, and monitor performance. Overall, Vivado streamlines the entire FPGA development cycle by offering automation, optimization, and deep visibility into hardware behaviour, making it suitable for both beginners learning digital design and professionals creating large, high-performance embedded systems.

3.4 SOFTWARE IMPLEMENTATION

Installation Procedure:

1. Visit the AMD/Xilinx Downloads page.
2. Choose the desired Vivado version (LTS or latest release).
3. Select the installer for your operating system.
4. Sign in with your Xilinx account (mandatory for all downloads).
5. Agree to the software license terms.
6. Choose between the Web Installer or Full Installer based on your internet speed and storage.
7. Download the file and launch the installer.
8. Select required FPGA device families during installation.
9. Install JTAG/USB cable drivers if supported by the OS.
10. Complete the installation and verify by launching Vivado.

For Windows:

1. Go to the official AMD/Xilinx website and open the “Downloads” section.
2. Select the Vivado version you want (typically the latest supported by your FPGA board
3. Choose the Windows installer (.exe file) under the Vivado WebPACK or full edition.
4. Create or log in to your Xilinx account to access the download.
5. Download either the Web Installer (small size) or the Full Installer (large size).
6. Once the file is downloaded, right-click the installer and choose “Run as Administrator.”
7. Accept the license agreements when prompted.
8. Select the installation directory and choose required device families (such as Artix-7)
9. Ensure the “Install Cable Drivers” option is enabled for JTAG programming.
10. Wait for the installer to download and install all components.
11. Restart the system if prompted.
12. Launch Vivado from the Start Menu and verify the installation.

CHAPTER 4

RESULTS AND DISCUSSIONS

4.1 PRESENTATION OF FINDINGS

The project was tested on the Basys3 FPGA board using a RISC-V processor with two different variants of ALU, one is a standard ALU and other is an ALU with Trojan injection. Test input sequences were used to observe the switching activity of both the ALUs using XOR and pop count logic provided at the RTL level. It was observed that the standard ALU operated within a relatively stable switching range of 4 to 10 transitions per cycle. On the other hand when the Trojan was activated the Trojan-injected ALU showed irregular spikes and often exceeded 15 transitions per cycle. This result shows that the hidden Trojan logic increases the dynamic power of the logic due to additional transitions.

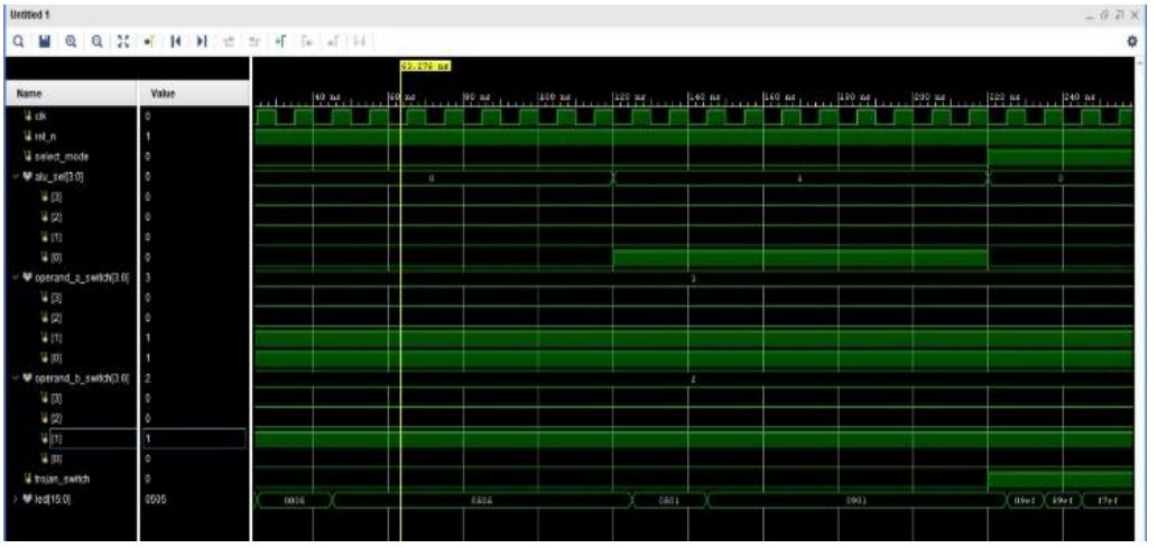

Figure 4.1: Waveform Output of Trojan-Free and Troja-Induced Circuit.

Power monitoring was done by checking the internal signal transition without any power measurement instruments. FPGA counted the switching events occurred in each clock cycle and if the count exceeded the predefined threshold, it set the detection flag. Table 4.1 shows the runtime monitoring of ALU in

RISC-V Processor. This system facilitated dependable, real-time detection without affecting functional output.

Mode	Input (e.g., A=3, B=2)	Output	Toggle Count	Trojan Alert (LED15)
Safe Mode	3 + 2	5	2 toggles	OFF
Infected Mode	3+2 Trojan= OFF	5	2 toggles	OFF
Infected Mode	3+2 Trojan= ON	DEADBEEF (Hex)	8+ toggles	ON

Table 4.1: Runtime Monitoring of ALU

4.2 POWER ANALYSIS RESULTS

The increase in switching count during Trojan activation proves the feasibility of using power based side channel analysis for security detection. The relation between bit transition and power consumption enabled the monitoring using internal signals without any physical power measurement.

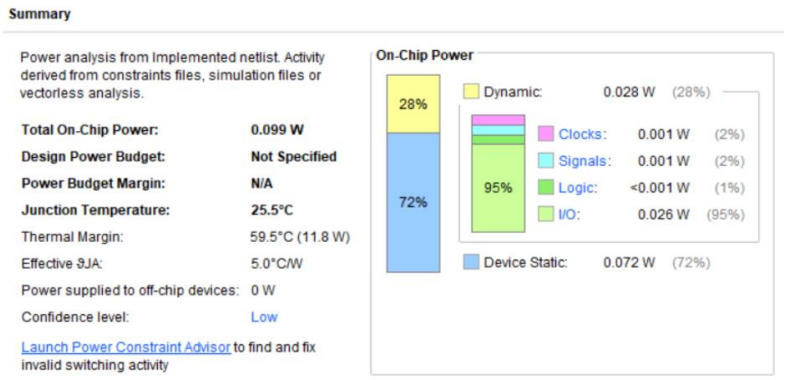

Figure 4.2 Normal ALU Report

The results of the normal ALU design are shown in Figure 4.3. Static power is 0.072 W (72%), dynamic power is 0.028 W (28%), hence the total on chip power consumption is 0.099 W. I/O uses 95% of the dynamic power, other blocks are not causing very less toggling, hence no abnormality is seen in their behaviour.

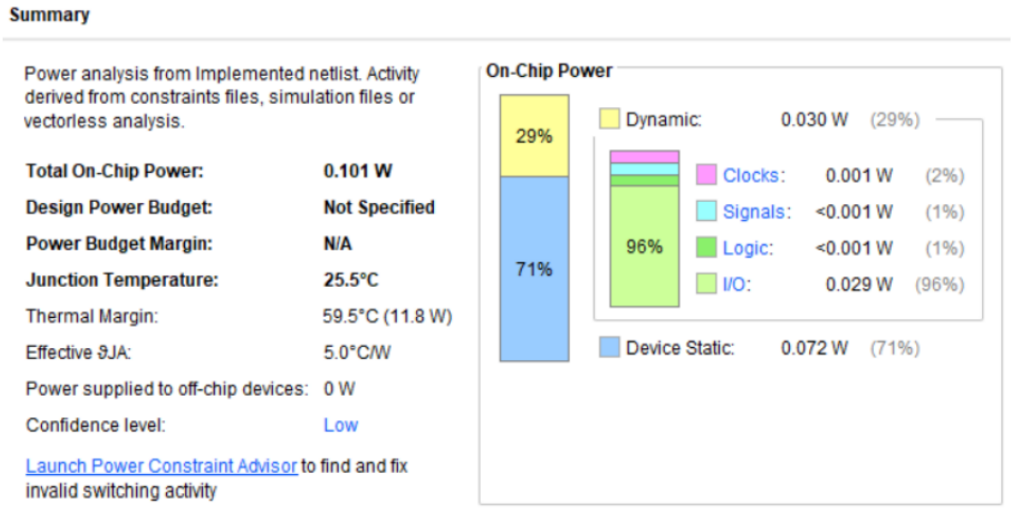

Figure 4.3 Trojan Infected ALU Report

The power analysis of the ALU design where Trojan is inserted is shown in Figure 4.3 Dynamic power increases to 0.030 W (29%), but the total on chip power increases slightly to 0.101 W. Unauthorized logic is seen by the discernible change in I/O power consumption and toggling (96%), which shows the effect of Trojan on the power behaviour of the circuit.

The amount of variation in power consumption caused by the Trojan is not very much to change the output behavior, but it was discernible enough to be monitored internally.

Table 4.2 compares power metrics of the Trojan-Free and Trojan-Infected ALUs. Due mainly to an increase in dynamic power and I/O power, the Trojan-Infected ALU exhibits a modest increase in total on-chip power from 0.099 W to 0.101 W. The little rise in I/O power (from 0.026 W to 0.029 W) suggests increased switching activity, which may be a symptom of a hardware Trojan, even though the device's static power stays constant at 0.072 W.

Metric	Trojan-Free ALU	Trojan-Infected ALU
Total On-Chip Power	0.099 W	0.101 W
Dynamic Power	0.028 W (28%)	0.030 W (29%)
Static_Power (Device Static)	0.072 W (72%)	0.072 W (71%)
I/O Power	0.026 W	0.029 W
Logic + Signal + Clocks	≈0.002 W	≈0.002 W

Table 4.2: Power Comparison

The fact that the logic, signal, and clock power are almost identical in all scenarios indicates that the Trojan has little effect on internal logic but raises activity on the exterior interface.

Additionally, the threshold-based detection mechanism proved both accurate and robust, with no false positives recorded during clean ALU testing. This suggests high reliability in distinguishing normal and abnormal conditions. The approach is also resource-efficient, requiring minimal additional logic, making it well-suited for low-power or embedded systems.

This study demonstrates that internal switching activity can serve as a proxy for power monitoring, and that such a technique is viable for detecting hidden threats in processor hardware. It opens up the possibility of integrating lightweight Trojan detection mechanisms into standard processor pipelines for enhanced runtime security.

4.3 FPGA IMPLENTATION RESULT

The implementation was successfully verified on the BASYS3 FPGA development board.

The board was programmed using Xilinx Vivado, and the desired functionality was observed through the state of onboard LEDs and 7-segment display outputs.

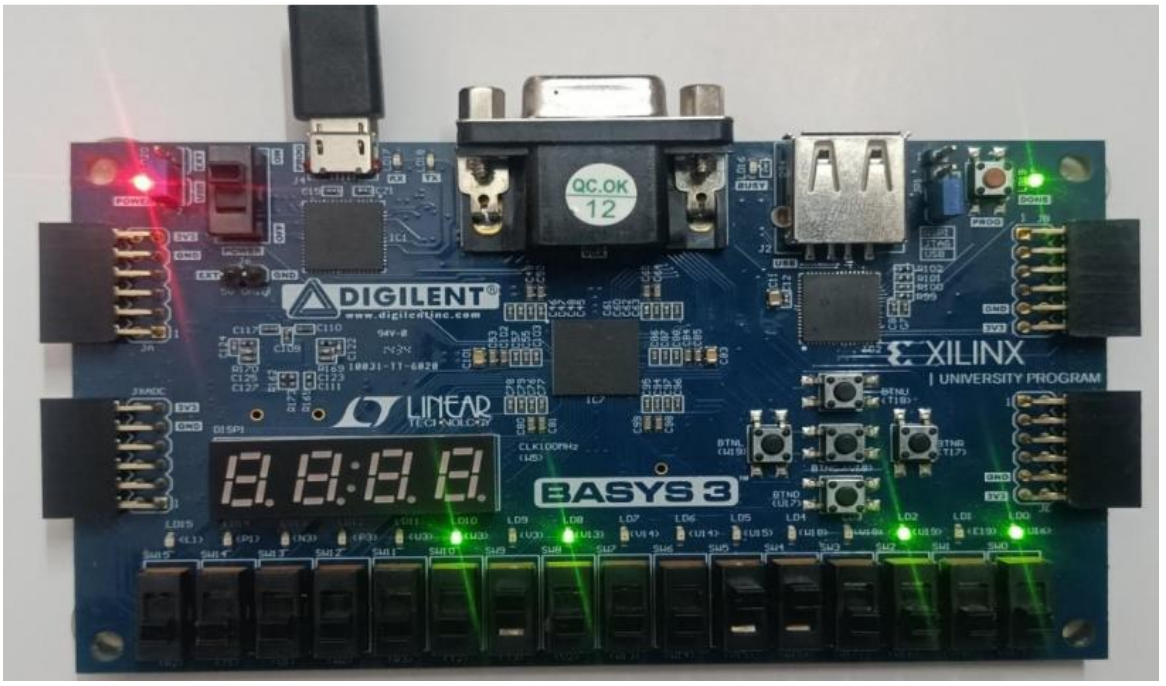

Figure 4.4 Trojan Free Mode

In this Trojan Free Mode, the ALU performs standard arithmetic and logic operations using operand inputs (set through switches SW4–SW11) and operation selection (SW0–SW3). The output appears on LEDs LD0–LD7, while LEDs LD8–LD14 show switching activity (bit toggle count) monitored via side-channel analysis.

As visible in the figure 4.4, only relevant output and power activity LEDs are ON, confirming normal functionality with no hidden or abnormal switching behaviour. LED15 (Trojan Alert) remains OFF, indicating a safe ALU operation.

Here, the Trojan is activated using the trojan_switch (BTN_U), and the ALU is manipulated to produce a malicious result (e.g., hardcoded DEADBEEF) instead of the true sum. The output LEDs show this invalid result.

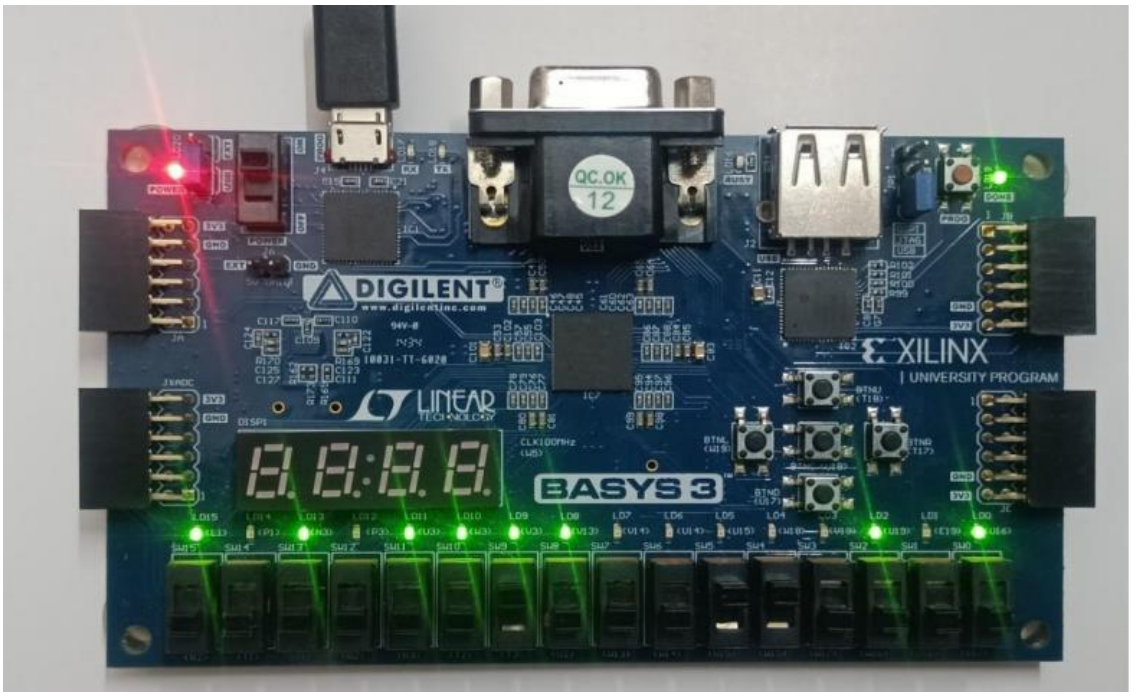

Figure 4.5 Trojan Infected Mode

Additionally, the switching activity (toggle count) significantly increases, which is detected by the signal_buffer and reported through higher values on LD8–LD14.Importantly, LD15 turns ON, confirming the detection of a Trojan condition either due to the abnormal result or high switching pattern flagged by the power monitor. This clearly demonstrates how the system detects both functional anomalies (wrong output) and side-channel anomalies (unexpected switching activity) using real-time FPGA logic.

4.4 Advantages

1. **High Reliability:**

   The use of safety and protection modules ensures continuous monitoring of faults, making the system highly reliable for motor control and power applications.

2. **Real-Time Operation:**

   FPGA-based implementation allows all modules SPWM generation, detection, current monitoring, and control logic to run in real time without delays.

3. **Modular Architecture:**

   Each block (SPWM, controller, safety, monitoring, detectors) is independent and well-defined, which makes the design easy to debug, expand, and upgrade.

4. **Accurate PWM Generation:**

   The SPWM processing unit generates precise and stable PWM signals which improve motor performance, reduce torque ripple, and enhance efficiency.

5. **Fast Fault Response:**

   The safety and current-monitoring blocks can detect abnormal conditions instantly and send immediate shutdown or control signals to prevent damage.

6. **Scalability:**

   Additional modules or external peripherals can be easily integrated through internal buses or FPGA I/O pins without redesigning the whole system.

7. **Flexible Control Modes:**

   The FSM-based controller supports multiple operating modes, allowing the system to adapt to different load conditions or user inputs.

8. **Efficient Resource Usage:**

   Synthesis and implementation in Vivado optimize FPGA resources (LUTs, flip-flops, BRAM, DSPs), leading to a compact and efficient hardware design.

4.5 Disadvantages

1. **Complex Design Process:**

   FPGA-based systems require knowledge of HDL coding, synthesis, timing analysis, and debugging, which increases development complexity.

2. **Higher Development Time:**

   Writing, simulating, synthesizing, and implementing multiple modules takes more time compared to microcontroller-based solutions.

3. **Resource Limitations:**

   Depending on the FPGA used (like the Basys 3 Artix-7), there may be limitations on available logic, BRAM, or DSP slices for advanced features.

4. **Higher Power Consumption than Simple Microcontrollers:**

   Although efficient, FPGAs typically consume more power than MCUs performing similar tasks.

5. **Cost Factor:**

   FPGA boards and tools can be more expensive compared to standard microcontroller development boards.

6. **Debugging Requires Tools:**

   FPGA debugging often requires tools like Integrated Logic Analyzer (ILA), which adds complexity compared to simple print-based debugging.

7. **Not Ideal for Extremely Large or AI-Based Designs:**

   Entry-level FPGAs may not support very large or computationally heavy designs unless upgraded to higher-end devices.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

This initiative effectively showcases a streamlined and efficient method for identifying Trojan threats within the ALU of a RISC-V processor through FPGA-based power analysis. By utilizing internal switching activity as an indirect indicator of dynamic power consumption, the system is capable of detecting irregular behaviour without the need for external power measurement devices. The implementation of XOR and pop count logic facilitated real-time monitoring of bit transitions, enabling the identification of subtle yet critical alterations induced by malicious logic. Experimental results confirmed that Trojan activity leads to tremendous switching activity increase, which was accurately detected and monitored by the system. The method is shown to be non-intrusive, energy efficient, and extremely reliable with no false alarms being identified under normal workloads. In summary, the method provides a cost-effective and scalable approach to hardware security enhancement for processor-based and embedded systems.

5.2 Future Scope

The future scope of this project offers several meaningful extensions that can significantly enhance its capability and applicability. The system can be expanded to interface with real-time data acquisition devices and monitoring dashboards, enabling continuous observation and documentation of Trojan-related activity for auditing and debugging purposes. The detection module may also be upgraded to support partial reconfiguration, allowing the system to dynamically switch between different detection algorithms based on specific application needs or environmental conditions. With further refinement, the FPGA-based solution can be adapted for deployment in critical domains such as defence electronics, aerospace systems, and secure industrial controllers, where hardware integrity and security are essential.

REFERENCES

[1] Rathor, V. S., Podder, S., & Dubey, S. (2025). An ensemble learning model for hardware Trojan detection in integrated circuit design. Computers and Electrical Engineering, 123, 110090.

[2] Zheng, T., Cai, G., & Huang, Z. (2022, May). A Soft RISC-V Processor IP with High-performance and Low-resource consumption for FPGA. In 2022 IEEE International Symposium on Circuits and Systems (ISCAS) (pp. 2538-2541). IEEE.

[3] Visoky, J., & Johnson, W. (2022, September). Industrial Protocol Security and Control System Performance: How Hardware Techniques Can Mitigate Performance Impacts of Cryptography. In Conference Proceedings of iSCSS (Vol. 2022).

[4] Kiaei, P., & Schaumont, P. (2022). SoC root canal! root cause analysis of power sidechannel leakage in system-on-chip designs. IACR Transactions on Cryptographic Hardware and Embedded Systems, 751-773.

[5] Ye, G., Shen, L., Feng, Y., Fan, L., Zhang, C., & Li, Y. (2024). Data-Driven-Based Event-Triggered Resilient Control for Cigarette Weight Control System Under Denial of Service Attacks and False Data Injection Attacks. IEEE Access.

[6] Patel, R., Parashar, H., & Wajid, M. (2011, March). Faster arithmetic and logical unit CMOS design with reduced number of transistors. In International Conference on Advances in Communication, Network, and Computing (pp. 519-522). Berlin, Heidelberg: Springer Berlin Heidelberg.

[7] Patyra, M. J., & Braun, E. (1996, September). Fuzzy/Scalar RISC processor: architectural level design and modeling. In Proceedings of IEEE 5th International Fuzzy Systems (Vol. 3, pp. 1937-1943). IEEE.

[8] Lamech, C., Rad, R. M., Tehranipoor, M., & Plusquellic, J. (2011). An experimental analysis of power and delay signal-to-noise requirements for detecting Trojans and methods for achieving the required detection sensitivities. IEEE Transactions on Information Forensics and Security, 6(3), 1170-1179.

[9] Rathor, V. S., Podder, S., & Dubey, S. (2025). An ensemble learning model for Trojan detection in integrated circuit design. Computers and Electrical Engineering, 123, 110090.

[10] Sunkavilli, S., Zhang, Z., & Yu, Q. (2021, July). New security threats on fpgas: From fpga design tools perspective. In 2021 IEEE Computer Society Annual Symposium on VLSI (ISVLSI) (pp. 278-283). IEEE

[11] Elnaggar, R., Chaudhuri, J., Karri, R., & Chakrabarty, K. (2022). Learning malicious circuits in FPGA bitstreams. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 42(3), 726-739.

[12] Chaudhuri, J., & Chakrabarty, K. (2022, May). Detection of malicious FPGA bitstreams using CNN-based learning. In 2022 IEEE European Test Symposium (ETS) (pp. 1-2). IEEE.

[13] Cruz, J., Posada, C., Masna, N. V. R., Chakraborty, P., Gaikwad, P., & Bhunia, S (2023). A framework for automated exploration of trojan attack space in FPGA netlists. IEEE Transactions on Computers, 72(10), 2740-2751.

[14] Samyukta, K., & Ramesh, S. R. (2023, October). Detection of Hardware Trojan Horse using Unsupervised Learning Approach. In 2023 IEEE International Conference on Distributed Computing, VLSI, Electrical Circuits and Robotics (DISCOVER) (pp. 77-82). IEEE.

[15] Ghimire, A., Alkurdi, M., & Amsaad, F. (2024, January). Enhancing hardware trojan security through reference-free clustering using representatives. In 2024 37th International Conference on VLSI Design and 2024 23rd International Conference on Embedded Systems (VLSID) (pp. 467-473). IEEE.

APENDIX

Trojan_free.v

```
module trojan_free(
 input wire clk,
 input wire rst,
 input wire [31:0] reg_x6_in,
 output reg [31:0] reg_x6
);
 always @(posedge clk or posedge rst) begin
 if (rst)
 reg_x6 <= 32'd0;
 else
 reg_x6 <= reg_x6_in + 1;
 end
endmodule
```

Trojan_infected.v

```
 module trojan_infected(
 input wire clk,
 input wire rst,
 input wire [31:0] reg_x6_in,
 input wire trojan_switch,
```

```
 output reg [31:0] reg_x6
);
 always @(posedge clk or posedge rst) begin
 if (rst)
reg_x6 <= 32'd0;
 else if (trojan_switch)
 reg_x6 <= 32'hDEADBEEF;
 else
 reg_x6 <= reg_x6_in + 1;
 end
endmodule
```

Trojan_Detector.v

```
module trojan_detector(
 input wire clk,
 input wire rst,
 input wire [31:0] reg_x6,
 output reg trojan_alert
);
 always @(posedge clk or posedge rst) begin
 if (rst)
 trojan_alert <= 0;
```

```
 else if (reg_x6 == 32'hDEADBEEF)
 trojan_alert <= 1;
 else
 trojan_alert <= 0;
 end
endmodule
```

Top_Module.v

```
module top_design_fpga(
 input wire
input wire rst,
 input wire select_mode,
 input wire [3:0] reg_x6_in_switch,
 input wire trojan_switch, // Now controlled by SW4 (W13)
 output wire [15:0] led
);
 wire [31:0] reg_x6_safe, reg_x6_infected;
 wire [31:0] reg_x6;
 wire [31:0] reg_x6_in;
 wire trojan_alert;
 assign reg_x6_in = {28'b0, reg_x6_in_switch};
 trojan_free safe_inst (
```

```
 .clk(clk),
 .rst(rst),
 .reg_x6_in(reg_x6_in),
 .reg_x6(reg_x6_safe)
 );
 trojan_infected infected_inst (
 .clk(clk),
 .rst(rst),
 .reg_x6_in(reg_x6_in),
 .trojan_switch(trojan_switch), // Controlled by SW4
 .reg_x6(reg_x6_infected)
 );
 assign reg_x6 = (select_mode) ? reg_x6_infected : reg_x6_safe;
assign led[14:0] = reg_x6[14:0];
 assign led[15] = trojan_alert;
 trojan_detector detect_inst (
 .clk(clk),
 .rst(rst),
 .reg_x6(reg_x6),
 .trojan_alert(trojan_alert)
 );
```

Endmodule

Constraints .xdc

```
## CLOCK - 100MHz (W5)

set_property PACKAGE_PIN W5 [get_ports clk]

set_property IOSTANDARD LVCMOS33 [get_ports clk]

create_clock -period 10.0 -name sys_clk_pin [get_ports clk]

## RESET - Center Button (BTNC - W19)

set_property PACKAGE_PIN W19 [get_ports rst]

set_property IOSTANDARD LVCMOS33 [get_ports rst]

## SELECT MODE - Down Button (BTND - T18)

set_property PACKAGE_PIN T18 [get_ports select_mode]

set_property IOSTANDARD LVCMOS33 [get_ports select_mode]

## TROJAN SWITCH - SW4 (W13)

set_property PACKAGE_PIN W13 [get_ports trojan_switch]

set_property IOSTANDARD LVCMOS33 [get_ports trojan_switch]

## SWITCHES for reg_x6_in_switch[3:0]

set_property PACKAGE_PIN R2 [get_ports {reg_x6_in_switch[0]}]

set_property PACKAGE_PIN V17 [get_ports {reg_x6_in_switch[1]}]

set_property PACKAGE_PIN V16 [get_ports {reg_x6_in_switch[2]}]

set_property PACKAGE_PIN W16 [get_ports {reg_x6_in_switch[3]}]

set_property IOSTANDARD LVCMOS33 [get_ports {reg_x6_in_switch[*]}]
```

```
## LED OUTPUTS - LD0 to LD15

set_property PACKAGE_PIN L1 [get_ports {led[0]}]

set_property PACKAGE_PIN U16 [get_ports {led[1]}]

set_property PACKAGE_PIN E19 [get_ports {led[2]}]

set_property PACKAGE_PIN U19 [get_ports {led[3]}]

set_property PACKAGE_PIN V19 [get_ports {led[4]}]

set_property PACKAGE_PIN W18 [get_ports {led[5]}]

set_property PACKAGE_PIN U15 [get_ports {led[6]}]

set_property PACKAGE_PIN U14 [get_ports {led[7]}]

set_property PACKAGE_PIN V14 [get_ports {led[8]}]

set_property PACKAGE_PIN V13 [get_ports {led[9]}]

set_property PACKAGE_PIN V3 [get_ports {led[10]}]

set_property PACKAGE_PIN W3 [get_ports {led[11]}]

set_property PACKAGE_PIN U3 [get_ports {led[12]}]

set_property PACKAGE_PIN P3 [get_ports {led[13]}]

set_property PACKAGE_PIN P1 [get_ports {led[14]}]

set_property PACKAGE_PIN N3 [get_ports {led[15]}]

set_property IOSTANDARD LVCMOS33 [get_ports {led[*]}
```